BIG DATA ANALYTICS · CÓMPUTO MASIVO

Estimación de π por Cuasi-Monte Carlo

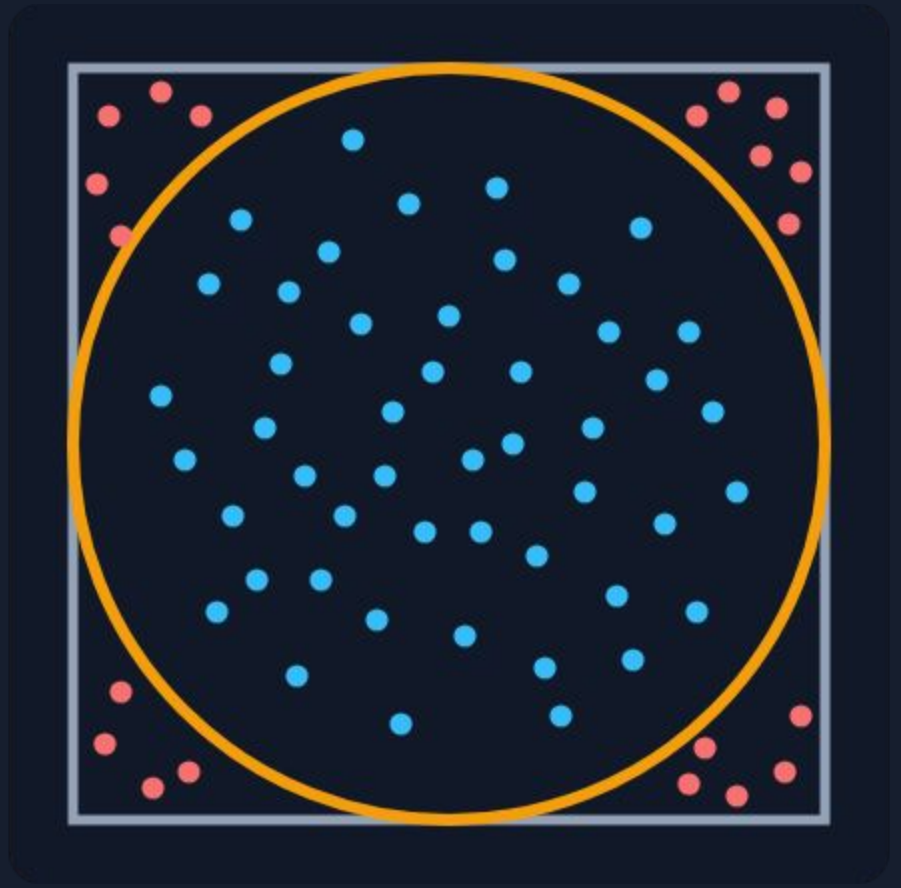
Práctica de Cómputo Distribuido en Apache Hadoop MapReduce (Standalone)

Equipo 4

Gustavo Adalid Catalán Carmen

Valentín Almaraz Martínez

Alberto Saúl López Crespo



CONTEXTO EN BIG DATA

Introducción al Problema y Contexto en Big Data

El reto del cómputo intensivo

Problemas científicos, financieros y físicos sin fórmulas cerradas que requieren millones de simulaciones numéricas iterativas.

¿Por qué MapReduce?

Más allá de wordcount o grep: plataforma para cómputo avergonzantemente paralelo con tareas 100% independientes.

Casos de aplicación industrial

- **Mercados financieros:** Valuación de derivados y opciones (Black-Scholes y pruebas de estrés).
- **Física nuclear y medicina:** Simulación de dispersión de neutrones y radioterapia.
- **Ingeniería avanzada:** Estimación de áreas y volúmenes multidimensionales complejos.

GEOMETRÍA Y PROBABILIDAD

Fundamento Matemático de Monte Carlo

$$\frac{A_{\text{círculo}}}{A_{\text{cuadrado}}} = \frac{\pi / 4}{1} \Rightarrow \pi \approx 4 \times \frac{\text{puntos dentro}}{\text{puntos totales}}$$

Principio geométrico: Círculo de radio r = 0.5 inscrito en un cuadrado unitario de lado L = 1.

Relación de áreas: A_cuadrado = 1^2 = 1 · A_círculo = π r^2 = π $(0.5)^2$ = π / 4.

Condición algebraica: Para cada punto (x, y), cae dentro si cumple x^2 + y^2 ≤ r^2.

FLUJO DE DATOS Y TRAZA REALIZADA POR EL EQUIPO

Arquitectura MapReduce y Nuestro Ejercicio: pi 2 5

1. INPUT

(offset, size)

offset = i · nSamples

→

2. FASE MAP

Evalúa Halton

(true, n_dentro) / (false, n_fuera)

→

3. SHUFFLE & SORT

Agrupa Claves

true → [d1, d2] · false → [f1, f2]

→

4. REDUCE & 5. SALIDA

Suma y Multiplica

π ≈ 4 × dentro / total

NUESTRO EJERCICIO PASO A PASO DEMOSTRADO: PI 2 5 (2 MAPPERS × 5 MUESTRAS = 10 PUNTOS)

Map #0 (Puntos 1 a 5)

5 dentro, 0 fuera

`Emite (true, 5) y (false, 0)`

+

Map #1 (Puntos 6 a 10)

4 dentro, 1 fuera

`Emite (true, 4) y (false, 1)`

→

Reduce (Suma Total)

9 dentro, 1 fuera (10 pts)

`π ≈ 4 × 9 / 10 = 3.6`

CLAVE TÉCNICA DEL ALGORITMO

Cuasi-Monte Carlo vs. Pseudoaleatorios

Monte Carlo Tradicional

Generador común: Función estándar Math.random() estocástica.

Distribución irregular: Forma cúmulos densos y regiones vacías.

Convergencia lenta: Tasa de error decrece a razón $O(1/\sqrt{N})$.

No reproducible: Cambia en cada ejecución según semilla.

Secuencia de Halton (Hadoop)

Baja discrepancia: Secuencia determinista en bases primas 2 y 3.

Cobertura uniforme: Llena el plano bidimensional simétricamente.

Convergencia superior: Velocidad cercana a orden $O(1/N)$.

Determinismo total: Precisión idéntica de 20 decimales.

FLUJO DE DATOS DISTRIBUIDO

Arquitectura MapReduce de pi

1. ENTRADA

Pares (offset, size)

Asigna segmento i · nSamples por Mapper.

2. FASE MAP

Puntos de Halton

Emite (true, dentro) y (false, fuera).

3. SHUFFLE

Agrupación Booleana

Consolida listas por clave booleana.

4. REDUCE Y SALIDA

Cálculo Final

Suma conteos aplicando 4 · dentro / total.

Ejemplo pi 2 5 (10 puntos): Map #0 emite (true, 5) · Map #1 emite (true, 4) y (false, 1) → Reduce: 4 × 9 / 10 = 3.6

ARQUITECTURA DE SISTEMA

Infraestructura y Entorno en Docker

- **Contenedor Base:** Entorno aislado sobre Debian 11 Bullseye reproducible.
- **Motor MapReduce:** Apache Hadoop 3.3.6 en modo Standalone (LocalJobRunner).
- **Tiempo de Ejecución:** Máquina virtual Java OpenJDK 11 Headless.
- **Procesamiento en Memoria:** Genera puntos en RAM sin archivos de entrada físicos.
- **Limpieza Automática:** Directorio temporal QuasiMonteCarlo_* se elimina al concluir.

Demostración Técnica y Comando de Ejecución

```
# 1) Sintaxis formal de ejecución en contenedor
hadoop jar
$HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar pi
10 1000
# 2) Automatización con scripts/ejecutar.sh (detecta /opt/hadoop o
/home/hadoop/hadoop-3.3.6)
bash /tmp/equipo4/ejecutar.sh # Genera /tmp/equipo4/resumen.txt y
/tmp/equipo4/logs/
```

Resultado de consola esperado:

```
Number of Maps = 10 | Samples per Map = 1000
Starting Job ... Job Finished in X seconds
Estimated value of Pi is 3.14080000000000000000
```

3.1408

10,000 puntos (10 × 1,000)

7 corridas automatizadas por nuestro script desde 10 hasta 1,000,000 de puntos

RESULTADOS EXPERIMENTALES

Análisis de Precisión y Convergencia

Parámetros	Puntos Totales (N)	π Estimado (20 decimales)	Error Absoluto
`pi 2 5`	10	`3.60000000000000000000`	0.45840
`pi 10 10`	100	`3.20000000000000000000`	0.05840
`pi 10 100`	1,000	`3.14800000000000000000`	0.00640
`pi 10 1000`	10,000	`3.14080000000000000000`	0.00079
`pi 10 10000`	100,000	`3.14120000000000000000`	0.00039
`pi 10 100000`	**1,000,000**	`3.14155200000000000000`	0.00004

Valor real de π = 3.14159265 · Al llegar a 1 millón de muestras el error cae más de 10,000 veces.

Mini-Ejercicio de Clase: ¿Qué tan bueno es tu π?

Reto práctico en contenedor: `1) pi 2 5 · 2) pi 10 10 … pi 10 100000 (error < 0.0005)`

1. Reparto de carga

`pi 1 1000 | pi 10 100 | pi 20 50`

¿Qué diferencia de resultado hay entre ejecutar pi 1 1000, pi 10 100 y pi 20 50?

2. Determinismo

`pi 10 1000 (3 veces seguidas)`

Si corremos pi 10 1000 tres veces seguidas, ¿por qué el resultado es idéntico cada vez?

3. Escalabilidad

`16 Mappers en 1 sola PC`

¿Por qué no usamos siempre 16 mappers si estamos en una sola computadora personal?

Solución y Discusión del Ejercicio

3.1480

Respuesta 1: Reparto de carga

Los tres comandos dan exactamente **3.148** porque el total de puntos (**1,000**) y el recorrido de la secuencia de Halton son iguales. Solo cambia cómo se fragmenta el trabajo entre tareas Map.

Determinista

Respuesta 2: Sin azar

No existe aleatoriedad estocástica; la secuencia de Halton (bases 2 y 3) es **determinista** y no depende de semillas variables del reloj del sistema.

Overhead JVM

Respuesta 3: Escalabilidad

En modo Standalone de una sola máquina, crear demasiados mappers introduce **sobrecarga (overhead)** de inicialización de JVM y gestión de contexto. El paralelismo masivo requiere un clúster físico distribuido.

CONCLUSIONES · EQUIPO 4

Cómputo Científico Reproducible en MapReduce

MapReduce resuelve simulaciones numéricas masivas sin I/O de disco mediante contenedores Docker estandarizados.

`github.com/valentinalmaraz401-dot/bda-equipo-4-pi`

¿Preguntas o comentarios?